In [10]:
```
# %pip install pandas numpy matplotlib seaborn scikit-learn scipy openpyxl p
```

In [11]:
```
# %pip install opencv-python
```

In [ ]:
```
# 3)
# Write a program to solve a fractional Knapsack problem using a greedy
# method. 01
# 4)
# Write a program to solve a 0-1 Knapsack problem using dynamic programming
# or branch and bound strategy.
# 02
# 5)
# Design n-Queens matrix having first Queen placed. Use backtracking to plac
# remaining Queens to generate the final n-queen's matrix. 01
# 6)
# Write a program to implement dictionary using hashing. (Use separate
# # chaining)
```

In [10]:
```
def fractional_knapsack(weights,profits, capacity):
    items =[]

    for i in range (len(weights)):
        ratio = profits[i] / weights[i]
        items.append((ratio, weights[i], profits[i]))

    items.sort(reverse=True)

    total_profit = 0

    for ratio,weight,profit in items:

        if capacity >= weight:
            total_profit = total_profit + profit
            capacity = capacity - weight

        else:
            total_profit = total_profit + ratio*capacity
            break

    return total_profit

n = int(input("Enter the no. of items:"))

weights = []
profits = []

for i in range(n):

    weight = float(input(f"Enter the weight for {i+1}: "))
    profit = float(input(f"Enter the profit for{i+1}: "))

    weights.append(weight)
    profits.append(profit)
```

```
capacity = float(input(f"Enter the capacity: "))

answer = fractional_knapsack(weights, profits, capacity)

print("Maximum Profit:", answer)
```

```
Maximum Profit: 114.0
```

In [ ]: